

PLATAFORMA
INTELIGENTE PARA
RECRUITING ÓPTIMO DE
PERSONAL FREELANCE

Contenido

PLATAFORMA INTELIGENTE PARA RECRUITING ÓPTIMO DE PERSONAL FREELANCE

Contenido

I Introducción

1.1 Objetivos

Objetivo general

Objetivos:específicos

1.2 Audiencia

1.3 Justificación

II. Marco conceptual

2.1 Elementos clave

2.2 Ventajas

2.3 Desafíos

2.4 Ciclo de vida

2.5 Descripción del producto o tecnología

Funcionalidades

Beneficios

Características

En resumen

III. Customer Journey

3.1 Descubrimiento

3.2 Secuencia de actividades

3.3 Wireframe

Descripción

3.4 UI

Ver figma completo

3.5 Arquitectura del Proyecto

Componentes

Despliegue

IV. Metodología de desarrollo

4.1 Introducción

4.2 Implementación de la Metodología August

V. Desarrollo de la tecnología

5.1 Introducción

5.2 Diseño y implementación

Frontend

Backend

1. lógica del servidor Parse Server y Express:

Características principales de Parse Server:

Beneficios de usar Parse:

2. Bases de datos y almacenamiento en el backend

Características principales de MongoDB:

[Beneficios de Mongo DB:](#)

[Base de datos](#)

[Modelo de Datos:](#)

[1. Diagrama de Datos y Esquema de la base de datos](#)

[2. Datos clave-valor y Optimización Redis](#)

[3. Políticas de Seguridad](#)

[Descripción de la arquitectura del backend:](#)

[Inteligencia Artificial](#)

[Datos Scrapeados:](#)

[Datos Simulados:](#)

[Proceso de Limpieza y Preprocesamiento de Datos](#)

[Código](#)

[Elección del Algoritmo de Machine Learning](#)

[Instanciación del Modelo:](#)

[Entrenamiento:](#)

[VI. Resultados y discusión](#)

[Front-end:](#)

[IA \(Inteligencia Artificial\):](#)

[Back-end:](#)

[Swagger:](#)

[VII. Conclusiones y recomendaciones](#)

[Recomendaciones:](#)

I Introducción

En un mercado laboral cada vez más competitivo, la búsqueda del talento ideal puede ser un desafío. Nuestro objetivo es revolucionar el reclutamiento con una plataforma inteligente que combina habilidades específicas y generales para evaluar a los candidatos, utilizando metodologías Data-Driven para calificar y clasificar automáticamente en función de sus resultados en pruebas diseñadas por especialistas y ejecutadas por inteligencia artificial. Con esta plataforma, los equipos de recursos humanos pueden basarse en datos confiables para tomar decisiones informadas sobre la selección de personal freelance.

1.1 Objetivos

Objetivo general

El objetivo general del proyecto es desarrollar una plataforma inteligente de reclutamiento que evalúe tanto habilidades específicas como generales de cada candidato, utilizando metodologías Data-Driven para calificar y clasificar automáticamente a los candidatos en función de sus resultados en pruebas diseñadas por especialistas y ejecutadas por inteligencia artificial. Esto permite al equipo de recursos humanos basarse en datos confiables para tomar decisiones importantes sobre la selección de personal freelance.

Objetivos:específicos

- Desarrollar un algoritmo de evaluación que analice y puntúe las habilidades específicas de los candidatos en áreas como programación, marketing digital y otras más relevantes para el proyecto.
- Implementar una interfaz de usuario intuitiva y amigable que permita a los empleadores crear pruebas personalizadas para evaluar a los candidatos.
- Integrar tecnologías de inteligencia artificial para ejecutar las pruebas diseñadas y recopilar datos sobre el rendimiento de los candidatos de manera automatizada.
- Diseñar un sistema de calificación automática que asigne puntajes objetivos a cada candidato en función de su desempeño en las pruebas y otros criterios definidos.
- Establecer una clasificación automática de los candidatos en función de los puntajes obtenidos en las pruebas, priorizando aquellos con las calificaciones más altas.
- Garantizar la seguridad y la privacidad de los datos de los candidatos durante todo el proceso de evaluación y clasificación.
- Realizar pruebas exhaustivas de la plataforma para identificar y corregir posibles errores o deficiencias en su funcionamiento.
- Ofrecer soporte técnico y capacitación adecuada al equipo de recursos humanos para utilizar eficazmente la plataforma y tomar decisiones informadas basadas en los datos proporcionados.

1.2 Audiencia

Este proyecto estaría dirigido a empresas u organizaciones que buscan reclutar personal freelance de manera eficiente y basada en datos. Estas empresas pueden pertenecer a diversos sectores, como tecnología, marketing, diseño, consultoría, entre otros, y pueden tener una amplia gama de necesidades de contratación de talento independiente para proyectos específicos. La plataforma inteligente de reclutamiento podría ser especialmente útil para equipos de recursos humanos que deseen optimizar el proceso de selección de personal freelance, asegurando la calidad y la idoneidad de los candidatos para las tareas requeridas.

1.3 Justificación

La justificación del proyecto se basa en mejorar la eficiencia, la precisión y la equidad del proceso de reclutamiento de personal freelance, lo que conduce a un beneficio tanto para las empresas empleadoras como para los candidatos independientes.

II. Marco conceptual

La metodología utilizada en este proyecto es la Metodología Integral de Desarrollo y Mejora Continua (August), un enfoque híbrido multidisciplinar que combina elementos de diferentes enfoques y disciplinas para lograr objetivos específicos. La August fue desarrollada por Dazlabs, y se basa en una combinación de prácticas y técnicas que han sido probadas y refinadas a lo largo del tiempo.

2.1 Elementos clave

- Combina elementos de Scrum: priorización del trabajo en proceso, retrospectiva regular y focus en la entrega de valor
- Combina elementos de Lean Development: enfocamiento en el progreso, eliminación de desperdicio y mejora continua
- Combina elementos de Kanban: visualización del flujo de trabajo, limitaciones en el flujo de trabajo y ajuste dinámico a los cambios
- Combina elementos de XP (Extreme Programming): testing continuo, refactorización y code review

2.2 Ventajas

- Flexibilidad para adaptarse a cambios en los requerimientos del proyecto
- Enfocamiento en la entrega de valor y mejora continua
- Visualización del flujo de trabajo y ajuste dinámico a los cambios
- Mejora de la comunicación y colaboración entre los equipos multidisciplinarios

2.3 Desafíos

- Integración de diferentes perspectivas y enfoques en un proceso único
- Gestión de las expectativas y priorización del trabajo en proceso
- Ajuste dinámico a los cambios en el proyecto y adaptabilidad al contexto

2.4 Ciclo de vida

La August se basa en un ciclo de vida que se cumple en el proceso de desarrollo, que consta de las siguientes fases:

1. Definición del objetivo: definir el objetivo y la estrategia para alcanzarlo
2. Planeación y priorización: planeación y priorización del trabajo en proceso
3. Desarrollo y testing: desarrollo y testing continuos
4. Refactorización y code review: refactorización y revisión del código
5. Entrega y retroalimentación: entrega de valor y retroalimentación continua

2.5 Descripción del producto o tecnología

Nuestra plataforma inteligente de reclutamiento es una herramienta innovadora que combina habilidades específicas y generales para evaluar candidatos, utilizando metodologías Data-Driven para calificar y clasificar automáticamente en función de sus resultados en pruebas diseñadas por especialistas y ejecutadas por inteligencia artificial. Esta plataforma revoluciona el proceso de reclutamiento de personal freelance, permitiendo a los equipos de recursos humanos basarse en datos confiables para tomar decisiones informadas.

Funcionalidades

- Desarrollar un algoritmo de evaluación que analice y puntúe las habilidades específicas de los candidatos
- Implementar una interfaz de usuario intuitiva y amigable para crear pruebas personalizadas
- Integrar tecnologías de inteligencia artificial para ejecutar las pruebas y recopilar datos sobre el rendimiento de los candidatos
- Diseñar un sistema de calificación automática que asigne puntajes objetivos a cada candidato
- Establecer una clasificación automática de los candidatos en función de los puntajes obtenidos
- Garantizar la seguridad y la privacidad de los datos de los candidatos

Beneficios

La plataforma inteligente de reclutamiento ofrece beneficios para las empresas empleadoras y los candidatos independientes.

Para las empresas:

- Mejora la eficiencia del proceso de reclutamiento
- Incrementa la precisión en la selección de personal freelance
- Asegura la calidad y la idoneidad de los candidatos
- Reduce el tiempo y los costos asociados con la búsqueda de talento

Para los candidatos:

- Ofrece una oportunidad más justa y equitativa para mostrar sus habilidades y experiencia
- Permite a los candidatos ser evaluados objetivamente en función de su desempeño
- Ayuda a los candidatos a encontrar proyectos que se adapten a sus habilidades y intereses

Características

La plataforma inteligente de reclutamiento es una herramienta intuitiva y amigable, con características como:

- Interfaz de usuario personalizable para crear pruebas
- Algoritmo de evaluación basado en metodologías Data-Driven
- Integración de tecnologías de inteligencia artificial para ejecutar las pruebas
- Sistema de calificación automática que asigne puntajes objetivos a cada candidato
- Clasificación automática de los candidatos en función de sus puntajes

En resumen, nuestra plataforma inteligente de reclutamiento es una herramienta revolucionaria que combina habilidades específicas y generales para evaluar candidatos, utilizando metodologías Data-Driven para calificar y clasificar automáticamente. Ofrece beneficios para las empresas empleadoras y los candidatos independientes, mejorando la eficiencia del proceso de reclutamiento, incrementando la precisión en la selección de personal freelance y asegurando la calidad y la idoneidad de los candidatos.

III. Customer Journey

3.1 Descubrimiento

La experiencia del cliente comienza con el descubrimiento de nuestra plataforma inteligente de reclutamiento freelance. Esta herramienta innovadora ofrece una solución avanzada para la selección y evaluación de candidatos, permitiendo a las empresas optimizar el proceso de reclutamiento. Al explorar nuestra plataforma, los usuarios pueden acceder a una interfaz intuitiva que facilita la creación de pruebas personalizadas y la ejecución automática de evaluaciones diseñadas por especialistas. Los empleadores descubren la capacidad de nuestra IA para calificar y clasificar candidatos de manera precisa, asegurando que solo los perfiles más adecuados sean seleccionados. En esta fase, las empresas y equipos de recursos humanos comienzan a apreciar los beneficios de una selección basada en datos, destacando la eficiencia, precisión y equidad en el proceso de reclutamiento.

3.2 Secuencia de actividades

1. Registro

- El usuario se registra en la plataforma con sus datos básicos (nombre, apellidos, correo electrónico, etc.).
- El usuario configura su perfil y establece preferencias y permisos según su rol (Freelancer o Reclutador).

2. Navegación y exploración

- El usuario explora la plataforma y descubre los diferentes recursos y funcionalidades.
- El usuario puede ver las diferentes secciones disponibles para la gestión de usuarios, generación de reportes, selección de postulantes y gestión de ofertas de empleo.

3. Gestión de usuarios

- El usuario selecciona la opción de gestionar datos.
- El usuario ingresa nuevos datos o actualiza los existentes.
- El sistema determina el rol del usuario (Freelancer o Reclutador).
 - Si es Freelancer, el usuario actualiza su portafolio, resumen curricular y aptitudes.
 - Si es Reclutador, el sistema almacena los datos ingresados y muestra una notificación de estado.

4. Generación de Reportes

- El usuario selecciona la opción de generación de reportes.
- El usuario selecciona el tipo de reporte que desea generar.
- El sistema consulta los datos necesarios y genera el reporte. El usuario recibe el reporte solicitado.

5. Proceso de Selección de Postulantes

- El Freelancer realiza la postulación a una oferta de empleo.
- El sistema almacena al postulante en la oferta de empleo.
- El Reclutador crea y ajusta la prueba de capacidades.
- El Reclutador envía la prueba de capacidades a los postulantes.
- El Freelancer completa la prueba.
- El sistema evalúa las pruebas recibidas y determina los mejores candidatos. El Reclutador selecciona al candidato final.
- El sistema marca como cerrada la oferta de empleo.
- El Freelancer recibe notificación de aceptación o rechazo.

6. Proceso de Selección de Postulantes

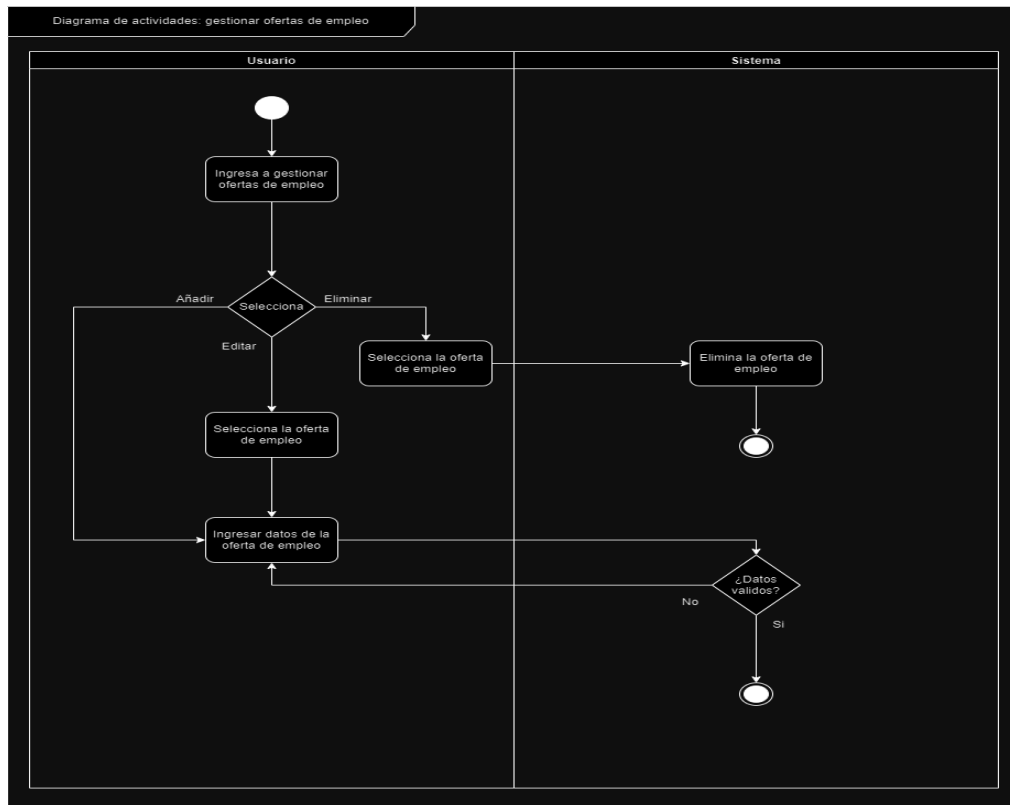
- El usuario accede a la sección de gestión de ofertas de empleo.
- El usuario selecciona la acción deseada (añadir, eliminar, editar).
- El usuario selecciona la oferta específica que desea gestionar.
- El sistema elimina la oferta de empleo (si es el caso).
- El usuario ingresa o edita los datos de la oferta de empleo.
- El sistema valida y almacena los datos de la oferta.

7. Proceso de Selección de Postulantes

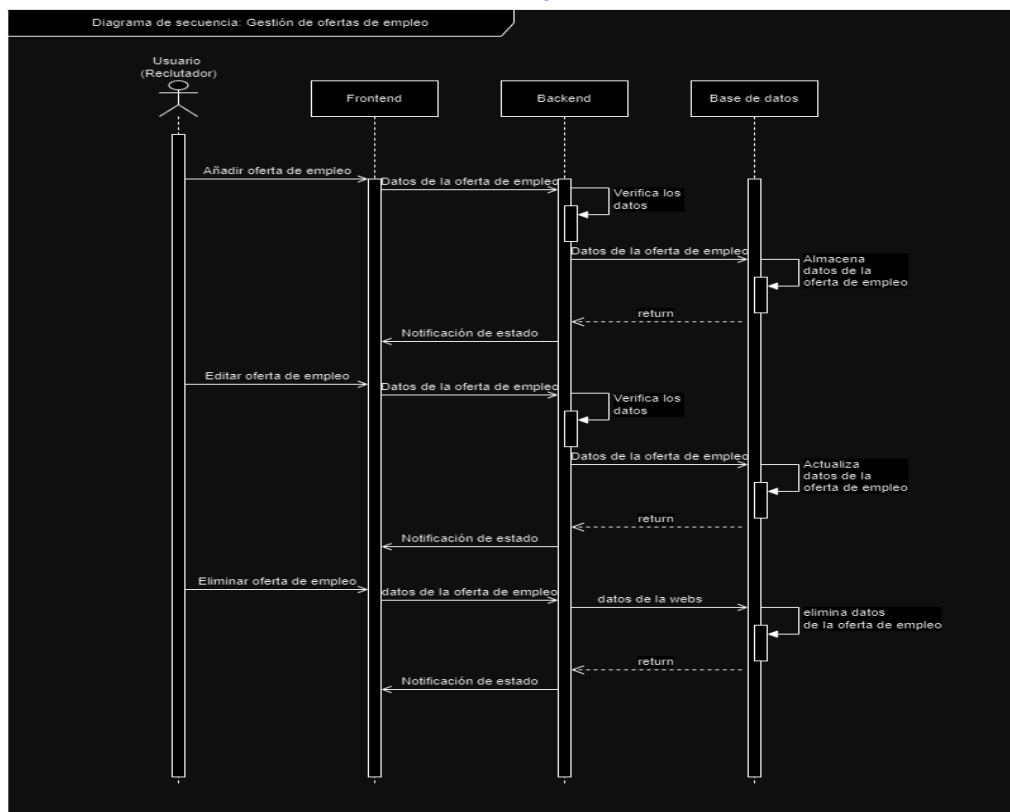
- El usuario comparte su contenido (ofertas de empleo, reportes, perfiles) con otros usuarios.
- El usuario puede recibir retroalimentación y colaborar con otros usuarios en la plataforma.

8. Proceso de Selección de Postulantes

- El usuario realiza la evaluación final de los datos ingresados y gestionados en la plataforma.
- El usuario puede exportar reportes, datos de usuarios y otros contenidos relevantes.



[Ver imagen](#)



[Ver imagen](#)

3.3 Wireframe

Página de llegada

- Cabecera con logotipo y menú de navegación
- Sección principal para mostrar recursos y funcionalidades
- Sección de registro y configuración del perfil

Panel de control admin

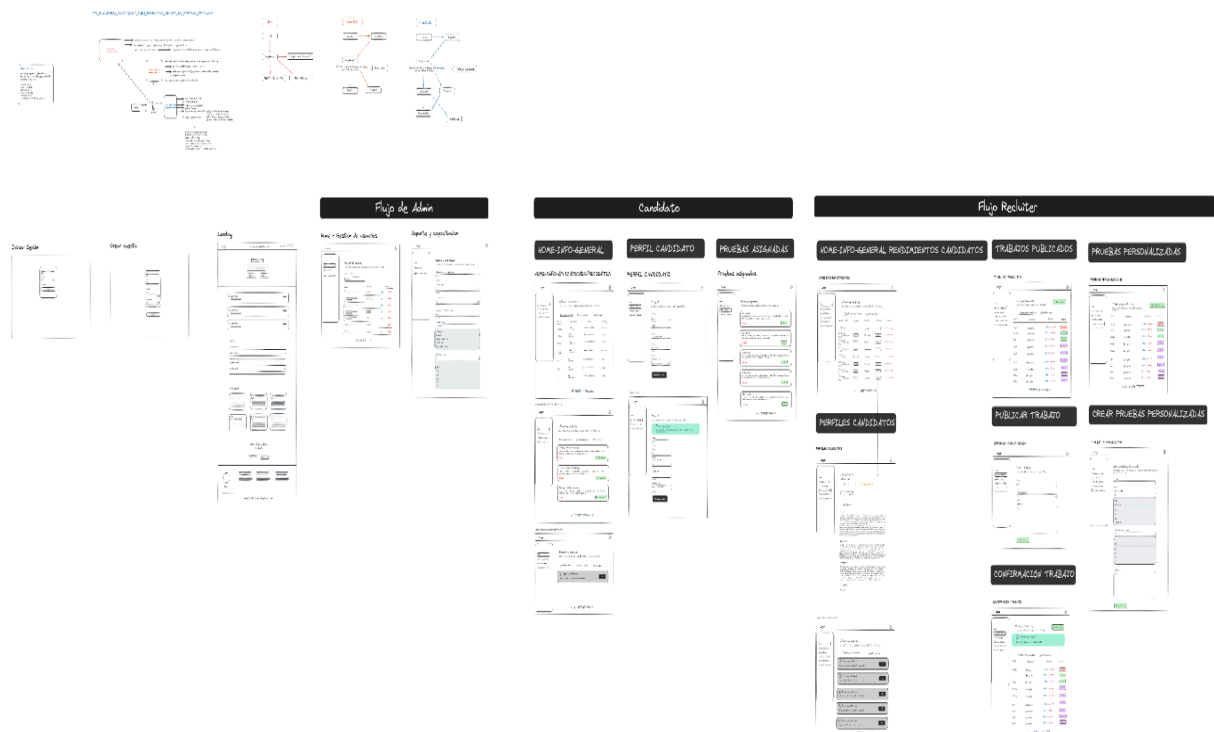
- Cargar capacitaciones
- Gestión de usuarios
- Herramientas

Panel de control de reclutador

- Perfil de los aplicantes
- Pruebas y resultados
- Publicar trabajos
- Notificaciones

Panel de control del candidato

- Perfil de la prueba
- Evaluaciones
- Notificaciones



[Ver imagen completa](#)

Descripción

La sección **"Descubrimiento"** comienza cuando el usuario conoce nuestra plataforma inteligente para el reclutamiento de personal freelance. En esta etapa, el usuario explora la propuesta de valor de nuestra herramienta, comprendiendo cómo optimiza el proceso de selección de candidatos mediante evaluaciones automatizadas y personalizadas. El usuario registra su interés y configura su perfil, estableciendo las bases para un proceso de reclutamiento eficiente.

En la sección **"Navegación y Exploración"**, el usuario explora la plataforma, familiarizándose con las funcionalidades disponibles, como la creación de pruebas personalizadas y la ejecución automática de evaluaciones. El usuario descubre cómo la IA clasifica y califica a los candidatos según sus resultados, proporcionando una visión detallada de las capacidades de la plataforma.

En la sección **"Configuración y Evaluación"**, el usuario crea y configura pruebas específicas para evaluar habilidades de candidatos freelance. Esta fase incluye el diseño de pruebas por parte de especialistas y la ejecución de estas mediante la inteligencia artificial, permitiendo una evaluación detallada y precisa de las habilidades de los candidatos.

En la sección **"Selección y Clasificación"**, el usuario revisa los resultados de las pruebas, recibe un informe detallado sobre el desempeño de los candidatos y utiliza esta información para tomar decisiones informadas en el proceso de selección.

Finalmente, en la sección **"Implementación y Soporte"**, el usuario integra los candidatos seleccionados en el proceso de contratación, accede a soporte técnico y capacitación para el equipo de recursos humanos, asegurando una transición fluida y eficiente desde la evaluación hasta la contratación.

3.4 UI

Implementación del wireframe en figma



[Ver figma completo](#)

3.5 Arquitectura del Proyecto

La plataforma inteligente para el recruiting óptimo de personal freelance se basa en un diseño de microservicios, que permite a cada servicio o aplicación ser desarrollada y escalada independientemente.

Componentes

- **API Gateway:** La puerta de entrada principal del sistema, responsable de recibir solicitudes y distribuirlas entre los diferentes servicios.
- **Render Service:** El servicio encargado de realizar la renderización del material audiovisual. Utiliza tecnologías como Rust con Python para implementar algoritmos de aprendizaje automático.
- **IA/ML Service:** El servicio que utiliza inteligencia artificial y machine learning para analizar el material audiovisual y predecir resultados.
- **Database Service:** El servicio responsable de almacenar y gestionar los datos del proyecto, incluyendo la información de proyectos, usuarios y versiones.
- **Storage Service:** El servicio que almacena y gestiona los archivos multimedia producidos por el sistema.
- **Frontend Service:** El servicio que se encarga de presentar la interfaz de usuario para los usuarios finales, utilizando tecnologías como React.
- **Worker Service:** El servicio que se ocupa de realizar tareas en segundo plano, como la renderización y procesamiento de archivos.

Despliegue

El proyecto se despliega en una infraestructura de nube pública, utilizando los servicios de Sinergy. Cada componente del sistema se encuentra en un contenedor Docker, lo que facilita el despliegue y la escalabilidad.



[Ver imagen completa](#)

IV. Metodología de desarrollo

4.1 Introducción

Metodología August

La metodología August es un enfoque integrado para el desarrollo de software que combina las prácticas de Scrum, Lean Development, Kanban y XP. Esta metodología se basa en la idea de que el desarrollo de software es un proceso iterativo e incremental que requiere flexibilidad, colaboración y retroalimentación continua.

La metodología August se caracteriza por:

- Iteraciones cortas y frecuentes (2-4 semanas) para reducir el riesgo y mejorar la velocidad de entrega
- Priorización de las características y requerimientos mediante el uso de votos y prioridades
- División del trabajo en tareas menores y autónomas para facilitar la colaboración y la responsabilidad
- Uso de Kanban boards para visualizar el flujo de trabajo y identificar oportunidades de mejora
- Implementación de técnicas de desarrollo ágil, como la programación en pairs, el coding review y el refactoring

El objetivo principal de la metodología August es mejorar la calidad, reducir el riesgo y aumentar la velocidad de entrega del software. Al combinar las prácticas de Scrum, Lean Development, Kanban y XP, esta metodología ofrece una estructura flexible y adaptable para el desarrollo de software.

4.2 Implementación de la Metodología August

En nuestra implementación de la metodología August, identificamos oportunidades para mejorar la eficiencia y la calidad de nuestro proceso de desarrollo. Nuestro objetivo era integrar las prácticas de Scrum, Lean Development, Kanban y XP en un enfoque flexible y adaptable.

La implementación de la metodología August comenzó con el equipo de diagramación, que generó un informe de requerimientos detallado y completo. Este documento se convirtió en el punto de partida para el equipo IA, que emitió una ruta analítica que indicaba la posible solución AI acompañada de las colecciones o campos de interés necesarios.

Luego, el equipo UX diseñó un wireframe que incluyó un diagrama modular del flujo de pantallas y la maquetación necesaria para el diseño de UI. Este documento se

revisó con los directores de equipo, lo que permitió ajustar ambos documentos (requerimientos y wireframe) para que los equipos inicien el desarrollo.

En este momento, cada equipo (UI, Front, Back y AI) comenzó a desarrollar su parte del proyecto en paralelo. El equipo UI se enfocó en la maquetación del proyecto, mientras que el equipo Front trabajaba en la maquetación de la interfaz de usuario. El equipo Back se centró en la arquitectura de back y el levantamiento de la base de datos, y el equipo AI desarrolló la técnica AI seleccionada.

Para garantizar la coordinación entre los equipos, implementamos un enfoque de micro-servicios, donde cada equipo desarrolla un servicio que luego se administra desde un backend as services llamado Parse Server. Una vez que cada equipo terminó su trabajo, comenzamos el acople del trabajo de cada uno y finalmente el despliegue de cada servicio y su interacción para la reproducción.

A través de esta implementación de la metodología August, logramos mejorar la comunicación entre los equipos, reducir la complejidad y aumentar la eficiencia en nuestro proceso de desarrollo.

Análisis de Beneficios y Desafíos

Beneficios:

- **Mejora en la comunicación:** La implementación de la metodología August permitió una mejor comunicación entre los equipos, lo que redujo la confusión y los errores.
- **Aumento de la eficiencia:** El uso de micro-servicios y el desarrollo paralelo de cada equipo permitió una mayor productividad y un aumento en la cantidad de trabajo realizado en un período determinado.
- **Flexibilidad y adaptabilidad:** La metodología August nos permite ser más flexibles y adaptarnos a los cambios y requerimientos del proyecto con facilidad.
- **Mejora en la calidad:** El enfoque en la programación en pairs, el coding review y el refactoring permitió una mayor calidad en el código y redujo el número de errores.

Desafíos:

- **Ajuste a la cultura organizativa:** La implementación de la metodología August requirió un ajuste significativo en la cultura organizativa, lo que puede ser un desafío para algunos miembros del equipo.
- **Reconocimiento y aceptación:** El cambio en el proceso de desarrollo puede generar resistencia en algunos miembros del equipo, lo que es importante abordar y comunicarse efectivamente.
- **Requisitos adicionales para el seguimiento y medición:** La implementación de la metodología August puede requerir el desarrollo de nuevos indicadores y métricas para medir el progreso y la efectividad del proceso.

V. Desarrollo de la tecnología

5.1 Introducción

El objetivo principal de este proyecto es desarrollar una plataforma inteligente para el recruiting óptimo de personal freelance. Esta herramienta de selección y reclutamiento de freelancers tiene como propósito evaluar las habilidades de los candidatos freelance para optimizar su selección, mejorando la eficacia y reduciendo los costos en el proceso de contratación.

En este capítulo, se presentará el desarrollo tecnológico detrás del proyecto, incluyendo la implementación de APIs, la creación de una base de datos robusta y la integración con terceros servicios. El objetivo es brindar una visión general detallada del proceso de desarrollo tecnológico que nos llevó a crear la plataforma.

La sección que sigue abordará los componentes clave del desarrollo tecnológico, seguida de un análisis de los desafíos tecnológicos importantes que enfrentamos durante el desarrollo del proyecto. Finalmente, se presentará el desarrollo tecnológico del proyecto en detalle, incluyendo el diseño y la implementación de los componentes clave.

En este proyecto, nos enfrentamos a varios desafíos importantes. Por ejemplo, debemos abordar la complejidad del modelo de BART, que requiere una gran cantidad de datos y un análisis exhaustivo de los patrones de comportamiento de los usuarios. Además, debemos asegurarnos de que el sistema sea escalable y pueda manejar un gran volumen de tráfico.

En la sección siguiente, describiremos en detalle el desarrollo tecnológico del proyecto, incluyendo el diseño y implementación de las componentes clave.

5.2 Diseño y implementación

Frontend

Utilizando React, este proyecto se beneficia de una biblioteca avanzada para aplicaciones web, ideal para crear una experiencia de usuario fluida y de alta interactividad en entornos virtuales.

Antes de hacer el commit correr en comando

```
yarn lint
```

para solucionar errores

Resumen de la estructura de carpetas

- **src/assets** : Carpeta para archivos estáticos, cómo imágenes, videos, etc.
- **src/schemas** : Carpeta para los **schemas** de las validaciones de yup o zod.
- **src/services** : Carpeta para las llamadas apis(en cuando usamos moralis) y también los endpoints de la API.

Ejemplo:

```
export function getIdata = async () => {  
  const response = await fetch(Services.auth.login)  
  const data = await response.json()  
  return data  
}
```

- **src/store** : Carpeta para las **stores** de la aplicación. Cuando las **stores** son demasiado grandes se podría mover las acciones a un archivo aparte cómo está en el ejemplo.
- **src/types** : Carpeta para los **tipos** de la aplicación. Se deben poner los tipos que son globales(que se utilizen el toda la app).
- **src/styles** : Carpeta para los **estilos**.
- **src/config** : Carpeta para las **variables de configuración** de la aplicación.
- **src/routes** : Carpeta para las **rutas del cliente** de la aplicación. Agregar las rutas en el archivo **index.ts**, cómo se muestra en el ejemplo y usarlo de la siguiente manera.

[Ver imagen](#)

React se destaca por su capacidad de construir interfaces de usuario eficientes y responsivas, lo que es crucial para cualquier plataforma que busca ofrecer una experiencia de usuario óptima. Su enfoque en la declaratividad y el renderizado eficiente asegura que los usuarios experimenten transiciones suaves y una interacción continua, sin demoras, al navegar por diferentes escenarios.

TypeScript para un desarrollo sólido: La implementación de TypeScript mejora la calidad del código y facilita la detección de errores, lo que es fundamental en el desarrollo de aplicaciones complejas. Esta combinación permite un desarrollo más robusto y mantenible, asegurando que la complejidad inherente a aplicaciones avanzadas se maneje de manera eficiente.

- Inicio
- Trabajos Publicitarios
- Publicar Trabajo
- Lista Candidatos
- Pruebas personalizadas
- Crear pruebas personalizadas

[Logout](#)

[Ver imagen](#)

- Inicio
- Trabajos Publicitarios
- Publicar Trabajo
- Lista Candidatos
- Pruebas personalizadas
- Crear pruebas personalizadas

[Logout](#)

[Ver imagen](#)

Nombre candidato

Frontend developer

HTML

CSS

Javascript



Armando Lios

example@gmail.com

Experiencia

Experiencia

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin sollicitudin purus et condimentum maximus. Cras fermentum, ex sed molestie aliquam, ligula lectus euismod odio, elementum maximus erat lacus nec diam. Pellentesque sodales risus ac nisi congue ultrices. Proin malesuada metus sed eros cursus, at tincidunt enim sagittis. Curabitur a tempor turpis, et aliquet velit. Vivamus ornare mauris nec viverra condimentum. Quisque enim diam, viverra vitae quam et, faucibus euismod mauris. Aliquam dapibus tellus nec est ullamcorper blandit. Donec imperdiet imperdiet leo id posuere. Duis commodo dolor rutrum, auctor tellus eu, fermentum dui. Mauris dignissim, nisi non molestie malesuada, mi diam aliquam massa, sed euismod nisi est maximus enim. Cras dignissim fermentum ullamcorper. Sed eget mattis ligula. Nulla pretium urna sed est congue, sit amet suscipit nisi pellentesque. Curabitur semper suscipit nulla, ac ullamcorper turpis convallis quis. Nunc id pellentesque ex. Suspendisse potenti. Nam ornare tellus at quam lacinia interdum. Maecenas ac sem porttitor mauris ultrices convallis nec eget justo. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Nullam quis pharetra metus, id pulvinar metus. Aliquam et interdum arcu. Fusce pharetra leo id enim aliquam, congue consequat mi egestas. Sed mollis dapibus tortor vitae viverra. Integer rhoncus erat tellus. Donec pretium sed mauris in tempus. Curabitur volutpat, tellus id venenatis facilisis, dui quam dapibus odio, id consequat quam tellus ut ante. Sed ornare urna et lorem molestie elementum.

Experiencia

It is a long established fact that a reader will be distracted by the readable content of a page when looking at its layout. The point of using Lorem Ipsum is that it has a more-or-less normal distribution of letters, as opposed to using 'Content here, content here', making it look like readable English. Many desktop publishing packages and web page editors now use Lorem Ipsum as their default model text, and a search for 'lorem ipsum' will uncover many web sites still in their infancy. Various versions have evolved over the years, sometimes by accident, sometimes on purpose (injected humour and the like).

Experiencia

There are many variations of passages of Lorem Ipsum available, but the majority have suffered alteration in some form, by injected humour, or randomised words which don't look even slightly believable. If you are going to use a passage of Lorem Ipsum, you need to be sure there isn't anything embarrassing hidden in the middle of text. All the Lorem Ipsum generators on the Internet tend to repeat predefined chunks as necessary, making this the first true generator on the Internet. It uses a dictionary of over 200 Latin words, combined with a handful of model sentence structures, to generate Lorem Ipsum which looks reasonable. The generated Lorem Ipsum is therefore always free from repetition, injected humour, or non-characteristic words etc.

Contacto

Email

Lista candidatos

Aquí puedes llevar como van tus postulaciones.

Rendimientos candidatos

Notificaciones

Tienes una notificación

Chequea tu postulación, nuevos candidatos.

ir

Tienes una notificación

Chequea tu postulación, nuevos candidatos.

ir

Tienes una notificación

Chequea tu postulación, nuevos candidatos.

ir

Tienes una notificación

Chequea tu postulación, nuevos candidatos.

ir

Tailwind CSS para una interfaz intuitiva y adaptativa: El uso de Tailwind CSS asegura una interfaz de usuario coherente y visualmente atractiva, lo que es vital para mantener a los usuarios comprometidos. Su diseño responsivo se adapta perfectamente a diferentes dispositivos, mejorando la accesibilidad y la experiencia de usuario en cualquier aplicación.

Vite.js para un desarrollo y construcción eficientes: Vite.js, seleccionado por su velocidad y eficiencia, se integra con TypeScript para proporcionar un entorno de desarrollo ágil. Esto permite un ciclo de desarrollo más rápido, con tiempos de carga instantáneos y feedback rápido para los desarrolladores, lo que es crucial para iterar rápidamente en el diseño y funcionalidad de cualquier plataforma.

Zustand para una gestión de estado eficiente: Zustand, seleccionado por su simplicidad y flexibilidad, se integra con React y TypeScript para proporcionar una gestión de estado ágil. Esto permite un manejo del estado global más intuitivo, con una API mínima y sin complicaciones para los desarrolladores, lo que es crucial para mantener la claridad y eficiencia en la lógica de estado de cualquier aplicación.

Yarn para la gestión eficiente de paquetes: La adopción de Yarn mejora el proceso de desarrollo al optimizar el almacenamiento y el rendimiento de la instalación de dependencias. Esto facilita el mantenimiento y la escalabilidad del proyecto, asegurando una base de código limpia y una integración sin problemas de nuevas tecnologías o bibliotecas.

Inicio

Trabajos Publicados

Publicar Trabajo

Lista Candidatos

Pruebas personalizadas

Crear pruebas personalizadas

Publicar trabajo

Aquí encuentras tus publicaciones para los candidatos.

Título

Título del Trabajo

Input helper text

Selecciona Puesto

Front End

Modifica tu CV

Lorem Ipsum

Publicar trabajo

Logout

[Ver imagen](#)

- Inicio
- Trabajos Publicitarios
- Publicar Trabajo
- Lista Candidatos
- Pruebas personalizadas
- Crear pruebas personalizadas

Logout

Lista candidatos

Aquí puedes llevar como van tus postulaciones.

Rendimientos candidatos

Notificaciones

Name / Email	Status	Position	Candidate	Actions	
 Armando Lios m@example.com	pending	Front End	ver	Avanza	No avanza
 Zoila Baca example@gmail.com	success	UX/ UI Designer	ver	Avanza	No avanza
 Armando Lios example@gmail.com	failed	Back End	ver	Avanza	No avanza
 Zoila Baca example@gmail.com	processing	UX/ UI Designer	ver	Avanza	No avanza
 Bienvenido Bueno example@gmail.com	failed	Front End	ver	Avanza	No avanza
 Armando Lios example@gmail.com	processing	Back End	ver	Avanza	No avanza
 Bienvenido Bueno example@gmail.com	success	Front End	ver	Avanza	No avanza
 Zoila Baca example@gmail.com	failed	Back End	ver	Avanza	No avanza

< Anterior

1

2

3

...

Siguiente >

[Ver imagen](#)

- Inicio
- Trabajos Publicados
- Publicar Trabajo
- Lista Candidatos
- Pruebas personalizadas
- Crear pruebas personalizadas

Logout

Trabajos publicados

Aquí encuentras tus publicaciones para los candidatos.

Publicar trabajo

Título	Descripción	Acciones		Estado
Back End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	pending
Front End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	success
Back End Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	failed
Front End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	processing
UX / UI Designer - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	failed
Back End Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	processing
UX / UI Designer - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	success
Front End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar	Eliminar	failed

< Anterior

1

2

3

...

Siguiente >

[Ver imagen](#)

Inicio

Trabajos Publicados

Publicar Trabajo

Lista Candidatos

Pruebas personalizadas

Crear pruebas personalizadas

Trabajos publicados

Publicar trabajo

Aquí encuentras tus publicaciones para los candidatos.

1

trabajo Nuevo Creado

Su nuevo trabajo se ha creado con éxito.

Título	Descripción	Acciones	Estado
Back End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	pending
Front End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	success
Back End Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	failed
Front End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	processing
UX / UI Designer - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	failed
Back End Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	processing
UX / UI Designer - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	success
Front End - Job	Lorem Ipsum is simply dummy text of the printing and typesetting industry	Editar Eliminar	failed

< Anterior 1 2 3 ... Siguiente >

Logout

[Ver imagen](#)

Fragmento de código en donde se aprecia la estructura de manejo de errores en el frontend

```
1  const createErrorFactory = (name: string, message?: string) => {
2    return class CustomizedError extends Error {
3      constructor(error?: Error) {
4        super(error?.message || message)
5        this.name = name
6        this.stack = ''
7      }
8    }
9  }
10
11  export const ConnectionError = createErrorFactory(
12    'ConnectionError',
13    'Conexión fallida. No se pudo establecer conexión con el servidor' "Conexión": Unknown word.
14  )
15  export const ValidationError = createErrorFactory('ValidationError')
16  export const AuthenticationError = createErrorFactory('AuthenticationError', 'Email o contraseña inválidos') "contraseña": Unknown word.
17  export const AuthorizationError = createErrorFactory('AuthorizationError', 'No tiene permisos suficientes') "tiene": Unknown word.
18  export const NotFoundError = createErrorFactory('NotFoundError', 'Página no encontrada') "Página": Unknown word.
19  export const TimeoutError = createErrorFactory('TimeoutError', 'Error de timeout. Intente de nuevo más tarde') "Intente": Unknown word.
20  export const ConflictError = createErrorFactory('ConflictError', 'Ya existe un usuario con esas credenciales') "existe": Unknown word.
21  export const ServerError = createErrorFactory('ServerError')
22  export const ClientError = createErrorFactory(
23    'ClientError',
24    'Parse recibió un JSON inválido. Puede haber errores de sintaxis, como comillas mal colocadas, comas faltantes, o falta de llaves de apertura o c
25  )
26  export const DatabaseError = createErrorFactory('DatabaseError')
27
```

[Ver imagen](#)

```

1 import {
2   ConnectionError,
3   ValidationError,
4   AuthenticationError,
5   AuthorizationError,
6   NotFoundError,
7   TimeoutError,
8   ConflictError,
9   ServerError,
10  ClientError,
11  DatabaseError,
12 } from './factory'
13
14 const errorHandler = (error: unknown) => {
15   if (error instanceof Error) {
16     if (error.message === 'XMLHttpRequest failed: "Unable to connect to the Parse API"') {
17       throw new ConnectionError()
18     }
19     if (error.message.includes('XMLHttpRequest failed:')) {
20       throw new ConnectionError(error)
21     }
22     if (error.message.includes('Received an error with invalid JSON from Parse:')) {
23       throw new ServerError(error)
24     }
25     if (error.message === '') {
26       throw new ValidationError()
27     }
28   }
29 }

```

[Ver imagen](#)

Fragmento de código en donde se aprecia el manejo encapsulado de Hooks, en este caso la imágenes muestran los hooks que usamos para el inicio de sesion del usuario

```

import { Moralis } from "moralis-v1";
import { useState } from "react";
import type { LogIn } from "../types";
import handleError from "@errors";

function useLogIn() {
  const [loading, setLoading] = useState(false);

  const logIn = async ({ email, password }: LogIn) => {
    setLoading(true);

    try {
      const response = await Moralis.User.logIn(email, password);
      if (response.id) {
        return { success: "Registro exitoso" };
      }
    } catch (error) {
      return handleError(error);
    } finally {
      setLoading(false);
    }
  };
}

```

```

    return {
      logIn,
      loading,
    };
  }

export default useLogIn;

```

En la siguiente imagen se muestra el componente con el formulario de autenticacion de usuario

```

12
13 export default function LogInForm() {
14   const { logIn, loading } = useLogIn()
15   const navigate = useNavigate()
16   const { showBoundary } = useErrorBoundary()
17   const formik = useFormik({
18     initialValues: {
19       email: '',
20       password: '',
21     } as LogIn,
22     validationSchema: LogInSchema,
23     onSubmit: async ({ email, password }) => {
24       try {
25         const result = await logIn({
26           email,
27           password,
28         })
29         if (result?.success) navigate(Routes.home)
30       } catch (error) {
31         return showBoundary(error)
32       }
33     },
34   })
35
36   return (
37     <main className="">
38       <div className="">
39         <span className="">Iniciar sesión</span>
40       </div>
41       <form className="" onSubmit={formik.handleSubmit}>
42         <div>
43           <label className="">
44             <span>Correo</span>
45             <input
46               onChange={formik.handleChange}
47               value={formik.values.email}
48               onBlur={formik.handleBlur}
49               name="email"
50               placeholder="email@gmail.com"

```

[Ver imagen](#)

En el siguiente fragmento de código mostramos el uso de la biblioteca yup para definir esquemas de validación para formularios de registro e inicio de sesión.

```
import * as yup from 'yup'
```



```

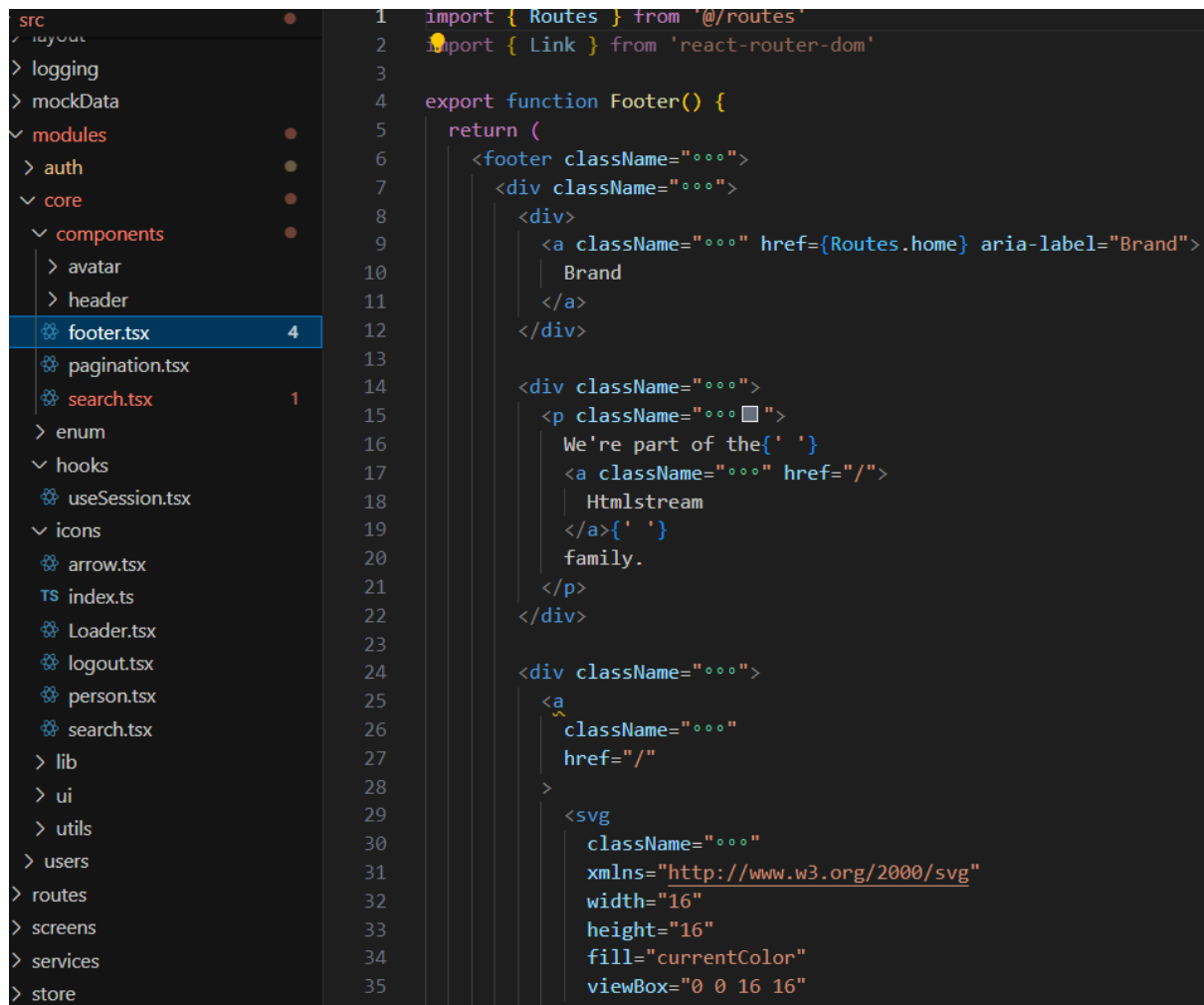
const passwordRules =
/^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#.%&@\$%^&\*])(?=.*{8,})/
const usernameSignUp = /^(S+$)/g
const emailRules = /^[^@]+@[^@]+\.[^@]+$

export const SignUpSchema = yup.object().shape({
  fullName: yup
    .string()
    .min(5, 'Full Name must be at least 5 characters long')
    .max(65, 'Full name must contain a maximum of 65 characters')
    .required('Required, Please Enter your Full Name'),
  username: yup
    .string()
    .min(5, 'Username must be at least 5 characters long')
    .max(25, 'User name must contain a maximum of 25 characters')
    .required('Required, Please Enter your User Name ')
    .matches(usernameSignUp, 'spaces not allowed'),
  email: yup.string().max(255).email('Must be a valid email').required('Email
is required'),
  password: yup
    .string()
    .required('Please Enter your password')
    .matches(
      passwordRules,
      'Must Contain 8 Characters, One Uppercase, One Lowercase, One Number and
One Special Case Character: ! @ # . * % & @'
    ),
})

export const LogInSchema = yup.object().shape({
  email: yup
    .string()
    .min(5, 'El nombre de usuario debe contener al menos 5 caracteres')
    .max(25, 'El nombre de usuario debe tener un máximo de 25 caracteres')
    .required('Requerido, por favor introduzca su nombre de usuario')
    .matches(emailRules, 'Debe ser un mail'),
  password: yup
    .string()
    .required('Requerido, por favor ingrese la contraseña')
    .matches(
      passwordRules,
      'Debe contener 8 caracteres, una mayúscula, una minúscula, un numero y
un carácter especial: : ! @ # . * % & @'
    ),
})

```

En la siguiente imagen podemos apreciar lo que es el core de la aplicación, en este módulo se manejan todos los componentes, hooks y utilidades que se reutilizan en diferentes partes de la aplicación.



```
1 import { Routes } from '@routes'
2 import { Link } from 'react-router-dom'
3
4 export function Footer() {
5   return (
6     <footer className="">
7       <div className="">
8         <div>
9           <a className="" href={Routes.home} aria-label="Brand">
10             Brand
11           </a>
12         </div>
13
14         <div className="">
15           <p className="">
16             We're part of the{' '}
17             <a className="" href="/">
18               Htmlstream
19             </a>{' '}
20             family.
21           </p>
22         </div>
23
24         <div className="">
25           <a
26             className=""
27             href="/"
28           >
29             <svg
30               className=""
31               xmlns="http://www.w3.org/2000/svg"
32               width="16"
33               height="16"
34               fill="currentColor"
35               viewBox="0 0 16 16"
36             />
```

[Ver imagen](#)

En el siguiente código se muestra un componente reutilizable que está dentro del core

```
import { useNavigate } from 'react-router-dom'
import { Search as SearchIcon } from '@modules/core/icons'

export default function Search() {
  const navigate = useNavigate()
  const url = new URL(globalThis.location.href)

  const handlerSubmit = (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault()

    const form = new FormData(event.currentTarget)

    const q = form.get('q') as string
```

```

    console.log(q)

    q?.length > 0 ? url.searchParams.set('q', q) :
url.searchParams.delete('q')

    navigate(url.pathname + url.search)
  }

  return (
    <form
      onSubmit={(e) => handlerSubmit(e)}
      className="bg-white shadow-lg border rounded-md max-w-lg w-full flex
gap-1 items-center py-1 pr-1 pl-3"
    >
      <input
        placeholder="Buscar"
        type="search"
        className="bg-transparent outline-none w-full"
        defaultValue={String(url.searchParams.get('q') || '')}
        name="q"
      />
      <button type="submit" className="bg-neutral-50 text-neutral-500
rounded-lg p-2">
        <SearchIcon />
      </button>
    </form>
  )
}

```

El siguiente código define un hook personalizado en React llamado useSession, que se utiliza para gestionar el estado de la sesión de usuario en la aplicación.

```

import { Routes } from '@routes'
import type { Session, SessionStatus } from '@types'
import Moralis from 'moralis-v1'
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { $SessionStatus } from '../enum'
// import { getUserById } from '@services/users'

function useSession() {
  const [user, setUser] = useState<Session | null>(null)
  const [status, setStatus] = useState<SessionStatus>($SessionStatus.LOADING)
  const navigate = useNavigate()

  useEffect(() => {
    try {

```

```

const currentSession = Moralis.User.current()

if (!currentSession?.attributes) {
  setStatus($SessionStatus.UNAUTHENTICATED)
  return
}

const { id } = currentSession

const { sessionToken, email, user_type } = currentSession.attributes

if (!sessionToken) {
  setStatus($SessionStatus.UNAUTHENTICATED)
  return
}

setUser({
  id,
  sessionToken,
  email,
  fullName: '',
  username: '',
  type: user_type,
})

// getUserById({ id }).then((response) => {
//   response.data &&
//     setUser({
//       id: currentSession.id,
//       sessionToken,
//       email: response.data?.email,
//       fullName: response.data?.fullName,
//       username: response.data?.username,
//       type: response.data?.type
//     })
// })

setStatus($SessionStatus.AUTHENTICATED)
} catch (error) {
  console.error(error)
  setStatus($SessionStatus.UNAUTHENTICATED_ERROR)
}
return () => {
  setUser(null)
}
}, [])

const signOut = () => {
  Moralis.User.logout()

```

```

    navigate(Routes.logIn)
    setStatus($SessionStatus.UNAUTHENTICATED)
  }

  return {
    user,
    status,
    signOut,
  }
}

export default useSession

```

En el siguiente código se muestra el manejo de rutas de la aplicación

```

import { useBoundStore } from '@store/index'
import { Navigate, Outlet } from 'react-router-dom'

export default function AdminRoutes({ redirectPath = '/log-in' }) {
  const { User, Authenticated, LoadingRefresh } = useBoundStore()

  if (LoadingRefresh) {
    return null
  }

  if (!Authenticated) return <Navigate to={redirectPath} replace={true} />

  // Verificar si User existe y si tiene la propiedad userType
  if (User?.user_type && User.user_type !== 'admin') {
    return <Navigate to={redirectPath} replace={true} />
  }

  return <Outlet />
}

```

```

import { useBoundStore } from '@store/index'
import { Navigate, Outlet } from 'react-router-dom'
export default function IsLogged({ redirectPath = '/log-in' }) {

```

```

const { Authenticated, LoadingRefresh } = useBoundStore()
if (LoadingRefresh) {
  return null
}

if (!Authenticated) {
  return <Navigate to={redirectPath} replace={true} />
}
return <Outlet />
}

```

Para la conexión con backend primero comenzamos ordenando los endpoints como se muestra en el siguiente código

```

export const Services = {
  auth: {
    signUp: 'createUser',
  },
  users: {
    update: 'updateUser',
    getUserById: 'getUserById',
    getAll: 'getUserOrDriver?type=user',
  },
  table: {
    find: 'findInTable',
  },
} as const

```

En el siguiente fragmento de código se muestra como hacemos el llamado a cada endpoint relacionado con los usuarios.

```

export const signUpUser = async ({ email, password, fullName, username }:
SignUp) => {
  try {
    const response = await Moralis.Cloud.run(Services.auth.signUp, {
      objectData: {
        email,
        password,
        username,
        fullName,
      },
    })

    if (response.status === 'error') {
      return {
        error: 'Ha ocurrido un error',
      }
    }

    return {

```

```

    success: 'Registro exitoso',
  }
} catch (error) {
  console.error(error)
  return {
    error: 'Ha ocurrido un error',
  }
}
}

```

En la siguiente imagen se muestra cómo gestionamos estados y acciones con Zustand.

```

import type { User } from '@types'

export type UserState = {
  User: User | null
  Authenticated: boolean
  LoadingUser: boolean
  LoadingRefresh: boolean
  LoadingUserEdit: boolean
}

export type UserActions = {
  setUser: (user: User) => void
  setAuthenticated: (value: boolean) => void
  setLoadingUser: (value: boolean) => void
  setLoadingRefresh: (value: boolean) => void
  setLoadingUserEdit: (value: boolean) => void
}

// biome-ignore lint/suspicious/noExplicitAny: <explanation>
export const useUserStore = (set: any) => ({
  User: null,
  ⚠Authenticated: false,
  LoadingUser: false,
  LoadingRefresh: true,
  UserById: null,
  LoadingUserEdit: false,

  setLoadingUser: (value: boolean) => set({ LoadingUser: value }),
  setUser: (user: User) => set(() => ({ User: user })),
  setAuthenticated: (value: boolean) => set({ Authenticated: value }),
  // biome-ignore lint/suspicious/noExplicitAny: <explanation>
  setUserById: (user: any) => set(() => ({ UserById: user })),
  setLoadingUserEdit: (value: boolean) => set({ LoadingUserEdit: value }),
  setLoadingRefresh: (value: boolean) => set({ LoadingRefresh: value }),
})

```

[Ver imagen](#)

Gracias a la adopción de la arquitectura Screaming Architecture en el lado del Frontend se garantiza la claridad, mantenibilidad y escalabilidad del código. Esta arquitectura asegura que la estructura de nuestro código grite su intención, haciendo que cada proyecto sea más fácil de entender y trabajar para cualquier desarrollador.

Backend

1. lógica del servidor Parse Server y Express:

Parse Server: Es un servidor de aplicaciones de código abierto que simplifica el desarrollo del backend para aplicaciones móviles y web. Funciona como un Backend como Servicio (BaaS), lo que significa que se encarga de la lógica y la gestión de datos del lado del servidor, liberando a los desarrolladores para que se concentren en la interfaz de usuario y la experiencia del usuario.

Características principales de Parse Server:

- **Base de datos en tiempo real:** Almacena y sincroniza datos en tiempo real, lo que permite que las aplicaciones se actualicen instantáneamente para todos los usuarios.
- **Autenticación de usuarios:** Proporciona un sistema seguro para autenticar y autorizar usuarios.
- **Almacenamiento de archivos:** Permite almacenar y administrar archivos de forma segura.
- **Notificaciones push:** Envía notificaciones push a los usuarios en sus dispositivos.
- **Código en la nube:** Ejecuta código JavaScript en el servidor para ampliar la funcionalidad de la aplicación.
- **Escalable:** Puede manejar un gran número de usuarios y solicitudes.
- **De código abierto:** Permite personalizar y adaptar el servidor a las necesidades específicas.

Beneficios de usar Parse:

- **Facilidad de uso:** Ofrece una API simple y fácil de usar, lo que la hace accesible para desarrolladores de todos los niveles.
- **Flexibilidad:** Es altamente personalizable y se puede adaptar a una amplia gama de aplicaciones.
- **Escalabilidad:** Puede manejar un gran número de usuarios y solicitudes, lo que la convierte en una buena opción para aplicaciones que esperan crecer.
- **Express:** Un marco web minimalista y flexible para Node.js que facilita la construcción de aplicaciones web y APIs. Es conocido por su simplicidad y velocidad, permitiendo el desarrollo de servidores HTTP.

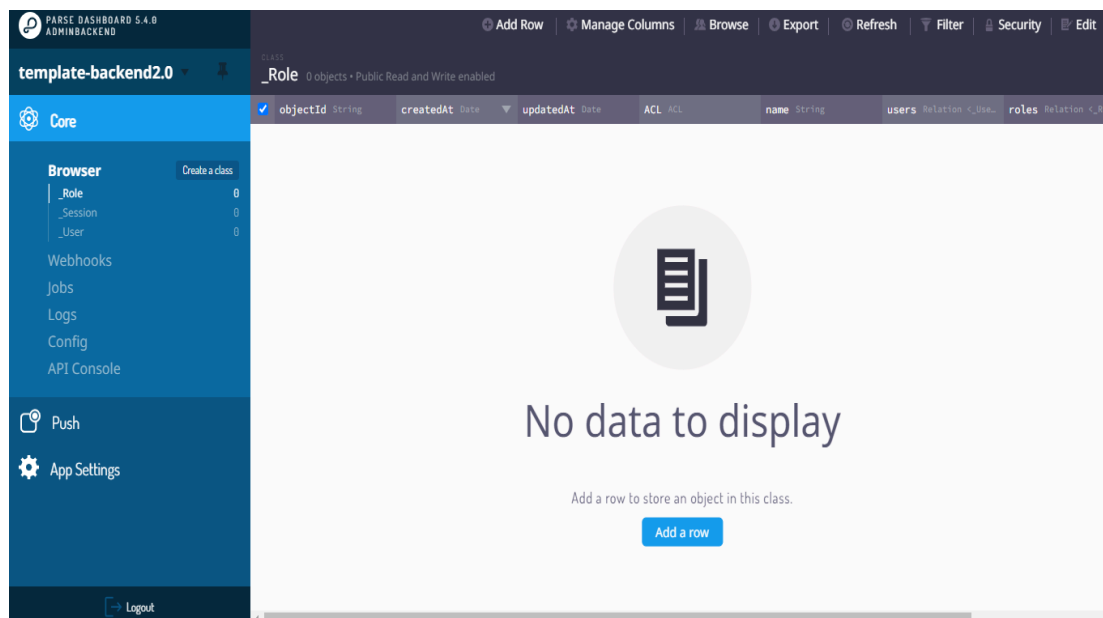
2. Bases de datos y almacenamiento en el backend

MongoDB: Es una base de datos NoSQL de código abierto, orientada a documentos y distribuida. En lugar de almacenar datos en tablas como las bases de datos relacionales tradicionales, MongoDB almacena datos en documentos JSON flexibles. Esto la hace ideal para almacenar y consultar datos no estructurados o semiestructurados, como datos de redes sociales, datos de sensores y datos de IoT.

Características principales de MongoDB:

- **Modelo de datos flexible:** Almacena datos en documentos JSON, lo que permite estructuras de datos no rígidas y dinámicas.
- **Esquema flexible:** No requiere un esquema predefinido, lo que permite agregar, eliminar y modificar campos de documentos fácilmente.
- **Consulta potente:** Soporta un lenguaje de consulta rico y flexible, que incluye consultas ad hoc, agregaciones y geoespaciales.
- **Escalabilidad horizontal:** Se puede distribuir en múltiples servidores para manejar grandes volúmenes de datos y tráfico.
- **Alto rendimiento:** Ofrece un rendimiento rápido y escalable para aplicaciones exigentes.
- **De código abierto:** Libre de usar y modificar, lo que permite la personalización y adaptación a necesidades específicas.

A continuación se muestra la base de datos de mongodb con el visor de parser-server



[Ver imagen de la base de datos](#)

Beneficios de Mongo DB:

- **Flexibilidad:** Es ideal para datos no estructurados o semiestructurados que no encajan bien en un modelo relacional.
- **Escalabilidad:** Puede manejar grandes volúmenes de datos y tráfico de forma eficiente.
- **Rendimiento:** Ofrece un rendimiento rápido y consistente para aplicaciones exigentes.
- **Facilidad de uso:** El modelo de datos flexible y el lenguaje de consulta intuitivo la hacen fácil de usar y aprender.
- **Comunidad grande y activa:** Cuenta con una comunidad grande y activa de desarrolladores que brindan soporte y comparten recursos.

Base de datos

Modelo de Datos:

En el caso de MongoDB y Parse Server, se utiliza un modelo de datos basado en documentos JSON. MongoDB es una base de datos NoSQL que almacena datos en documentos BSON (Binary JSON), y Parse Server utiliza MongoDB como su almacenamiento de datos predeterminado.

En este modelo, la información se organiza en documentos que son similares a objetos JSON, y estos documentos se almacenan en colecciones en lugar de tablas. Cada documento puede tener una estructura flexible, lo que significa que diferentes documentos en una misma colección pueden tener campos distintos.

1. Diagrama de Datos y Esquema de la base de datos

[Visitar documento](#)

2. Datos clave-valor y Optimización Redis

Caching de Consultas Frecuentes:

Parse Server identifica consultas o resultados de consultas que son frecuentemente utilizados pero no cambian con frecuencia.

Estos resultados se almacenan en Redis como pares clave-valor, donde la clave puede ser un identificador único asociado a la consulta y el valor es el resultado de la consulta.

Configuración de Parse Server:

Parse Server se configura para utilizar Redis como su sistema de almacenamiento en caché. Esto se hace mediante la configuración del parámetro `cacheAdapter` durante la inicialización.

Uso en Consultas:

Antes de realizar consultas a la base de datos principal, Parse Server verifica si los datos requeridos están presentes en la caché de Redis.

Si los datos están en la caché, se evita la consulta a la base de datos principal, mejorando significativamente el rendimiento.

Manejo de Expiración:

Parse Server configura tiempos de expiración para los datos almacenados en caché en Redis.

Esto asegura que los datos no estén obsoletos y se vuelvan a consultar de la base de datos principal periódicamente.

Reducción de Carga en la Base de Datos Principal:

Al almacenar en caché datos frecuentemente utilizados, Parse Server reduce la carga en la base de datos principal (por ejemplo, MongoDB).

Esto mejora el rendimiento general de la aplicación al disminuir la necesidad de realizar consultas intensivas en la base de datos.

Al implementar este enfoque de almacenamiento en caché con Redis, Parse Server logra una optimización significativa en términos de rendimiento al reducir la carga en la base de datos principal y al acelerar el acceso a datos frecuentemente utilizados.

3. Políticas de Seguridad

En Parse Server con MongoDB, las políticas de seguridad están relacionadas con cómo se controla el acceso a la base de datos MongoDB y cómo se aseguran los datos almacenados. políticas de seguridad en este entorno:

Autenticación en MongoDB:

MongoDB utiliza un sistema de autenticación basado en roles y usuarios. Parse Server debe autenticarse correctamente con MongoDB para acceder a los datos.

Las credenciales de acceso (nombre de usuario y contraseña) deben ser seguras y únicas para garantizar la autenticación segura entre Parse Server y MongoDB.

Autorización y Roles:

MongoDB permite la asignación de roles y permisos a los usuarios. Parse Server puede definir roles específicos para acceder a ciertos recursos o colecciones en MongoDB.

Es posible restringir las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) basadas en roles para garantizar que los usuarios tengan solo los permisos necesarios.

Configuración de Parse Server:

La conexión entre Parse Server y MongoDB se establece de manera segura, utilizando conexiones cifradas (SSL/TLS) para proteger la transferencia de datos, en producción.

Políticas de Seguridad de Parse Server:

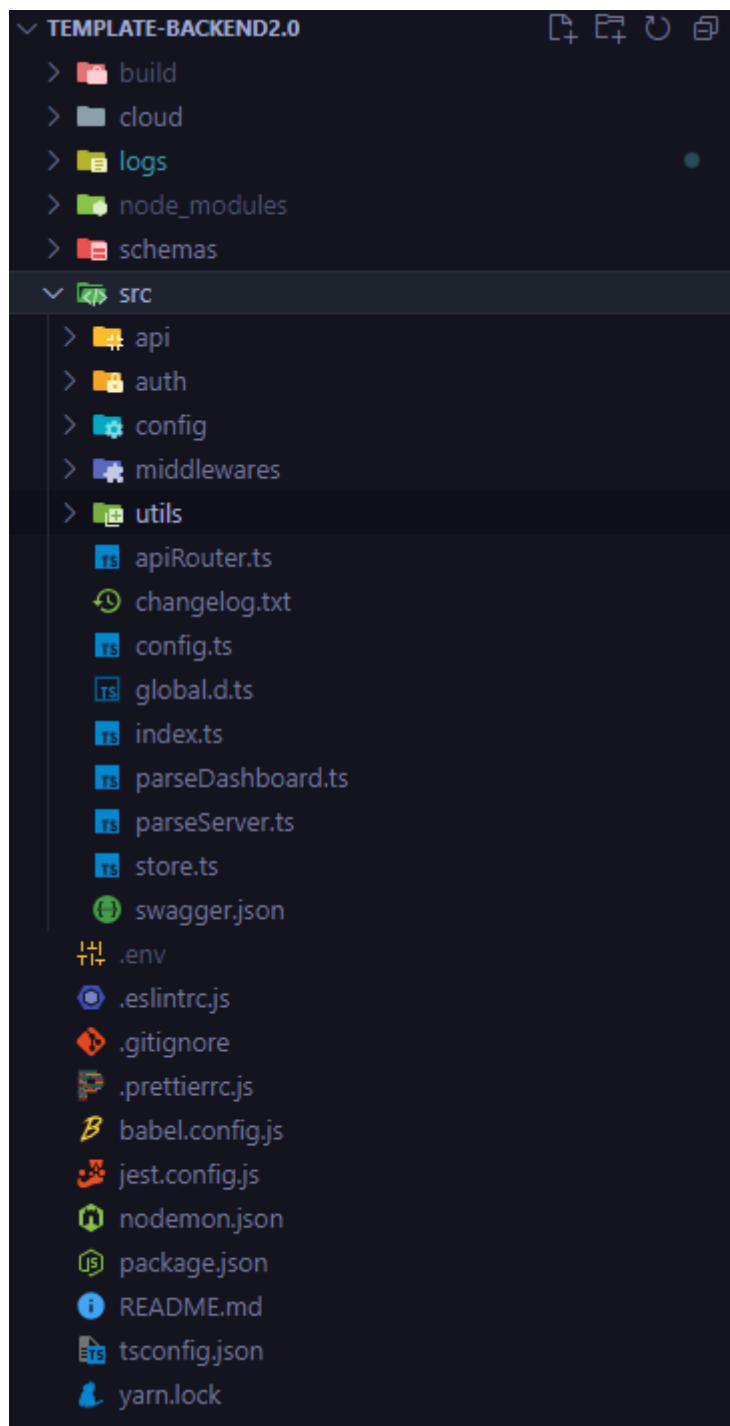
Parse Server implementa políticas de seguridad a nivel de aplicación, permitiendo definir quién tiene acceso a los datos y qué operaciones pueden realizar.

Se pueden configurar ACLs (Listas de Control de Acceso) en los objetos Parse para definir quién puede acceder, leer, escribir o eliminar esos objetos.

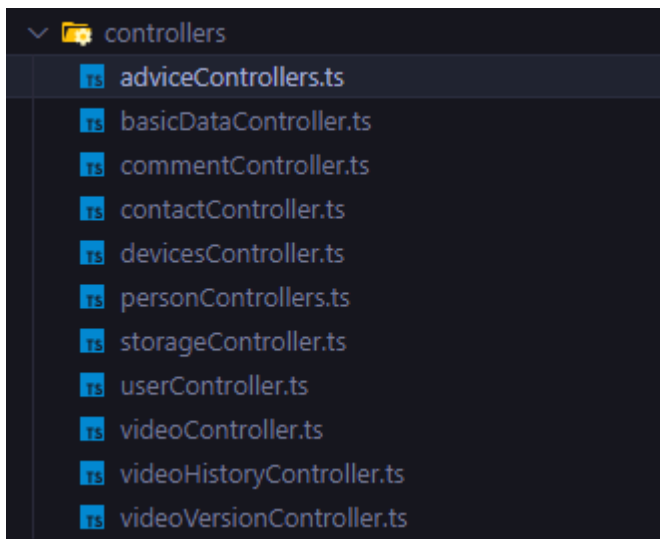
Descripción de la arquitectura del backend:

Se utiliza en esta versión una estructura hexagonal en donde tenemos el controlador que es a donde se recibe la petición del frontal , luego la capa de datos en donde se encuentra la lógica del negocio para después irnos a la capa de database en donde se almacena el objeto.

Arquitectura de Parser-server

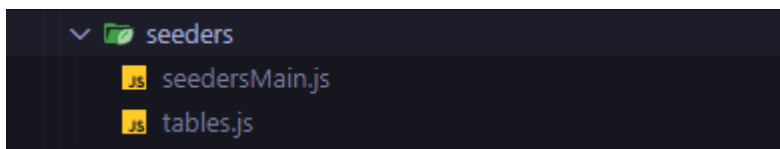


[ver imagen](#)



[Ver imagen](#)

En la imagen a continuación se observa la carpeta de los seeders que son los datos que nos pasa el departamento de IA para subir data a la base de datos y el frontal pueda mostrar información con la finalidad de demostrar el funcionamiento de la aplicación.



[Ver imagen](#)

```
import { loadAllData } from '../helpers/loadAllData';

export function seedersMain(Parse) {
  return async (request) => {
    try {
      await loadAllData();

      return {
        status: 'success',
        result: true,
        message: 'Seeding successfully completed.',
      };
    } catch (error) {
      console.error(`Código de error: ${error.code}, Mensaje de error: ${error.message}`);
      return {
        status: 'error',
      };
    }
  };
}
```

```

    result: false,
    errorDetails: {
      code: error.code || Parse.Error.INTERNAL_SERVER_ERROR,
      message: error.message,
    },
  },
};
}
};
}
}

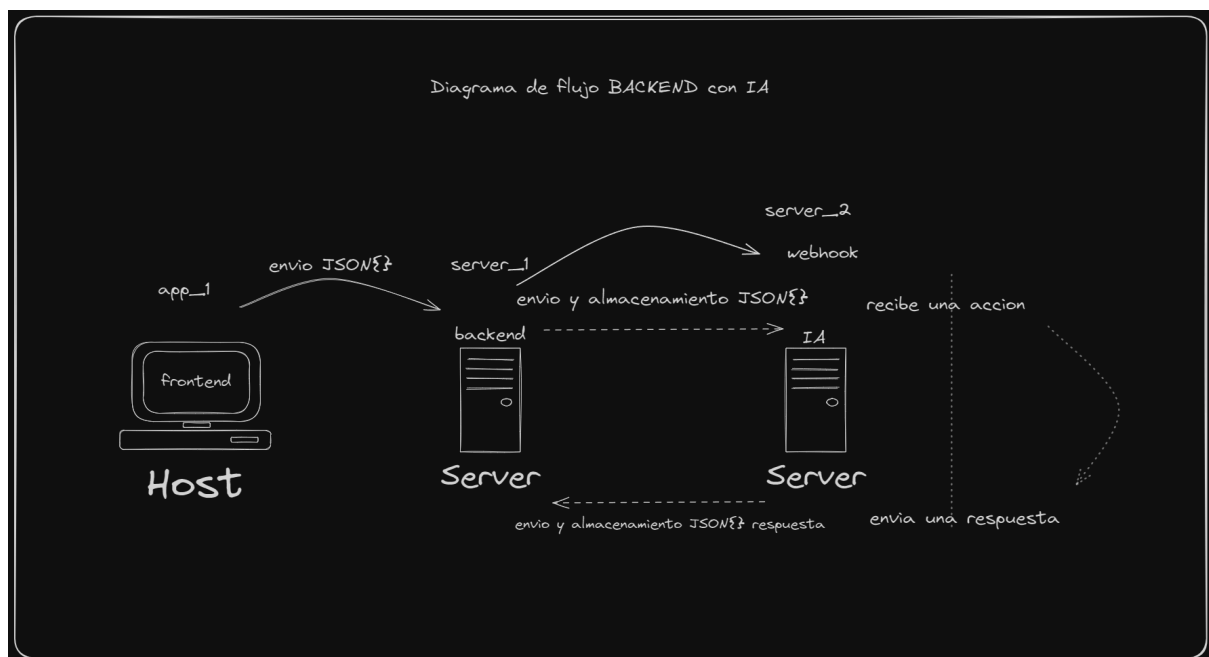
```

una vez que el proceso de validación se completa y los campos son correctos se pasa a la carpeta controller

4. Integración de IA mediante microservicios:

En este contexto, la integración de IA a través de microservicios implica dividir la lógica de inteligencia artificial en servicios independientes que se comunican a través de webhooks. Este enfoque permite una arquitectura más modular y escalable, donde la IA puede ser actualizada y escalada de manera independiente al resto de la aplicación.

Diagrama de Lógica de comunicación con IA:



5. Despliegue de servicios cloud computing

Sinergy Elastic Servers: Una opción de despliegue en la nube ofrecida por Sinergy que proporciona servidores virtuales escalables. Permite adaptarse a las necesidades

específicas del proyecto, con la capacidad de escalar automáticamente para manejar variaciones en la demanda.

Sinergy Apps Service: Un servicio de plataforma como servicio (PaaS) ofrecido por Sinergy que simplifica el proceso de implementación y gestión de aplicaciones. Permite a los desarrolladores centrarse en el código sin preocuparse por la infraestructura subyacente.

6. Seguridad y privacidad con Express y Parse-Server:

Ambas tecnologías ofrecen herramientas y funcionalidades para implementar medidas de seguridad, como autenticación y autorización. El uso de HTTPS es esencial para proteger las comunicaciones.

Inteligencia Artificial

A continuación, se proporciona una descripción detallada del proyecto en términos de inteligencia artificial (IA) centrado en el algoritmo BART (Bidirectional and Auto-Regressive Transformers). El proyecto combina estratégicamente datos scrapeados y datos simulados para construir un conjunto de datos completo y enriquecido que maximiza el alcance y la precisión de la evaluación de habilidades y la selección de candidatos freelance en la plataforma.

Origen y Naturaleza de los Datos

El conjunto de datos utilizado en este proyecto es el resultado de una combinación estratégica entre datos scrapeados y datos simulados. Esta fusión se llevó a cabo con el propósito de construir un dataset completo y enriquecido, capaz de proporcionar un alto nivel de precisión y alcance en las funcionalidades de evaluación de habilidades y selección de candidatos, asegurando así la diversidad y calidad de la información necesaria.

Datos Scrapeados:

Se obtuvieron mediante técnicas de web scraping de los siguientes sitios:

- [Visitar documento](#)

Datos Simulados:

Adicionalmente, se incorporaron datos ficticios simulados para completar las posibles brechas en la información y para proporcionar un conjunto de entrenamiento más robusto al modelo. Estos datos simulados se generaron siguiendo patrones realistas y escenarios que podrían surgir en el contexto de evaluación de habilidades y selección de candidatos, asegurando así la diversidad y calidad de la información necesaria

Generación de datos simulados con Faker para proyectos de IA

El proceso de generación de datos simulados para un proyecto de IA utilizando el módulo Faker.

- Definición de parámetros: Se definen los parámetros necesarios para la generación de datos, como el número de registros, el tipo de datos y los rangos de valores.
- Selección de campos: Se seleccionan los campos de la estructura de datos que se quieren generar.
- Generación de valores: Se utilizan las funciones de Faker para generar valores aleatorios para cada campo.
- Validación de datos: Se valida la calidad de los datos generados para garantizar que sean consistentes y representativos de los datos reales.
- Almacenamiento de datos: Se almacenan los datos generados en un formato adecuado, en este caso JSON.

Objetivos:

El objetivo principal de este proceso es generar datos simulados de alta calidad que puedan utilizarse para entrenar y evaluar modelos de IA, especialmente en situaciones donde no se dispone de datos reales suficientes o de calidad.

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.chrome.options import Options
4 import time
5 import pandas as pd
6 import json
7
8 class WebScraper:
9     def __init__(self, config_path):
10         with open(config_path, 'r') as file:
11             self.config = json.load(file)
12
13         options = Options()
14         options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36")
15         self.driver = webdriver.Chrome(options=options)
16         self.url = self.config["url"]
17         self.field_classes = self.config["field_classes"]
18         self.output_config = self.config["output"]
```

[ver imagen](#)

```

20     def getElementsByClass(self):
21         try:
22             self.driver.get(self.url)
23             time.sleep(10) # Esperar a que la página cargue
24             results = {}
25             for attr, class_name in self.field_classes.items():
26                 elements = self.driver.find_elements(By.CLASS_NAME, class_name)
27                 time.sleep(1) # Ajustar el tiempo de espera según sea necesario
28                 print(f'Se encontraron {len(elements)} elementos para {attr}')
29                 attr_values = [element.text for element in elements]
30                 results[attr] = attr_values
31
32             df = self.create_df(results)
33             self.obtainJsonOrExcelOrCsv(df)
34         except NameError as e:
35             print(f"Se produjo un error: {str(e)}")
36         finally:
37             self.driver.quit()

```

[ver imagen](#)

```

66     def main():
67         scraper = WebScraper('config.json')
68         scraper.getElementsByClass()
69
70     if __name__ == "__main__":
71         main()
72

```

[ver imagen](#)

```

1     {
2         "url": "nombre_url",
3         "field_classes": {
4             "key1": "className",
5             "key2": "className",
6             "key3": "className"
7         },
8         "output": {
9             "file_name": "fileName",
10            "extension": "json"
11        }
12    }

```

[ver imagen](#)

Estructura de datos:

La estructura de datos del proyecto plataforma para la recruiting óptimo de personal Freelance tiene los siguientes campos:

- **Nombre:** cadena de texto aleatoria (Nombre del postulante)
- **Puntuación:** Número entero aleatorio entre 1 y 50 (Número de años de experiencia)
- **Habilidades Técnicas y Blandas:** cadena de texto aleatoria (Habilidades técnicas y blandas que posee el freelancer..)
- **Experiencia en la Prueba Técnica:** número real aleatorio entre 0 y 100 (Puntuación de la prueba técnica)
- **Parametros de Preferencias:** cadena de texto aleatoria (Parametros de preferencias para un aspirante en la parte tecnica y blanda)
- **Empleo:** cadena de texto aleatoria (Nombre del empleo)
- **Aptitudes Requeridas:** cadena de texto aleatoria (Aptitudes requeridas de la oferta)
- **Estado de Evaluación:** cadena de texto aleatoria (Estado de pasada o no de las pruebas, lo cual puede ser un indicador de idoneidad para el puesto.)

Configuración de Faker:

En el código de ejemplo, **Configuración de Faker:**

Se crea una instancia de Faker utilizando la clase `faker.Faker()`. Esta instancia permite acceder a las funciones de Faker para generar valores aleatorios para diferentes tipos de datos.

Se crea una instancia de Faker utilizando la clase `faker.Faker()`. Esta instancia permite acceder a las funciones de Faker para generar valores aleatorios para diferentes tipos de datos.

Código Configuración

```
1 {  
2     "key1": "fakerFuncionNam    ,  
3     "key2": "fakerFuncionNam    ,  
4     "key3": "fakerFuncionNam    ,  
5     "key4": "    ,  
6     "key5": "random fieldNameOfAnotherColectio  
7 }
```

[ver imagen](#)

```

1 from faker_modul import FakerModule
2 import os
3 import json
4 import random
5
6 # Función para cargar la colección desde un archivo JSO
7 def cargar_coleccion(nombre_archiv ):
8     with open(nombre_archiv , 'r') as f:
9         return json.load(f)
10
11 # Función para exportar los datos falsos a un nuevo archivo JSO
12 def exportar_coleccio (coleccion, nombre_archiv ):
13     with open(nombre_archiv , 'w') as f:
14         json.dump(coleccion, f, indent=4, ensure_ascii=False)
15
16 # Cargar la configuración desde un archivo JSON
17 config = cargar_coleccion('./config.json')
18
19 # Crear una instancia del módulo Faker con la configuración cargada
20 faker_modul = FakerModule(config)
21
22
23 # Cargar la colección desde un archivo JSO
24 nombre_archivo_origina = './042 interests.json'
25 coleccion = cargar_coleccion(nombre_archivo_origina )
26
27 # Generar datos falsos extendido
28 datos_falso = faker_modul .extender_coleccio (coleccion, 1000)
29 # Por ejemplo, generar 1000 registros adicionales
30
31 # Construir el nombre del archivo exportado
32 nombre_base, extension = os.path.splitext(nombre_archivo_origina )
33 nombre_archivo_exportad = f"{nombre_base}_mod{extension}"
34
35 # Exportar los datos falsos a un nuevo archivo JSON con el nombre modificado
36 exportar_coleccio (datos_falso , nombre_archivo_exportad )
37
38 print(f"Los datos falsos han sido generados y guardados en {nombre_archivo_exportad }.")

```

[Ver imagen](#)

Proceso de Limpieza y Preprocesamiento de Datos

Los datos recolectados fueron sometidos a un proceso exhaustivo de limpieza y preprocesamiento para garantizar la calidad y coherencia de la información. Se llevaron a cabo las siguientes actividades:

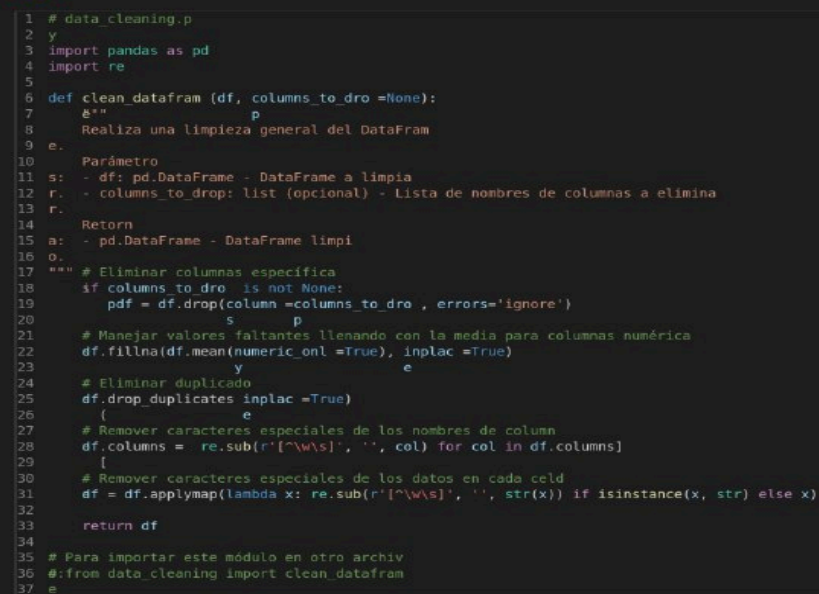
Limpieza de Datos

- **Manejo de Datos Faltantes:** Se identificaron y abordaron los valores nulos o faltantes de manera adecuada, ya sea mediante imputación o eliminación según el contexto.
- **Detección y Tratamiento de Outliers:** Se realizaron análisis estadísticos para identificar posibles valores atípicos que pudieran afectar la integridad de los resultados, y se aplicaron estrategias para gestionarlos.

Preprocesamiento.

- **Normalización y Escalado:** Las variables numéricas fueron normalizadas o escaladas según la necesidad del modelo, para garantizar una comparación adecuada entre diferentes magnitudes.
- **Transformación de Variables Categóricas:** Se realizaron conversiones y codificaciones de variables categóricas para integrarlas efectivamente en el modelo.

Código



```
1 # data_cleaning.py
2 y
3 import pandas as pd
4 import re
5
6 def clean_dataframe(df, columns_to_drop=None):
7     """
8     Realiza una limpieza general del DataFrame
9     e.
10     Parámetro
11     s: - df: pd.DataFrame - DataFrame a limpiar
12     r. - columns_to_drop: list (opcional) - Lista de nombres de columnas a eliminar
13     r.
14     Return
15     a: - pd.DataFrame - DataFrame limpi
16     o.
17     """
18     # Eliminar columnas específicas
19     if columns_to_drop is not None:
20         pdf = df.drop(columns=columns_to_drop, errors='ignore')
21         s
22         p
23     # Manejar valores faltantes llenando con la media para columnas numéricas
24     df.fillna(df.mean(numeric_only=True), inplace=True)
25     y
26     # Eliminar duplicados
27     df.drop_duplicates(inplace=True)
28     e
29     # Remover caracteres especiales de los nombres de columna
30     df.columns = re.sub(r'[\W\s]', '', col) for col in df.columns
31     {
32     # Remover caracteres especiales de los datos en cada celda
33     df = df.applymap(lambda x: re.sub(r'[\W\s]', '', str(x)) if isinstance(x, str) else x)
34     }
35     return df
36
37 # Para importar este módulo en otro archivo
38 #:from data_cleaning import clean_dataframe
39 e
```

[Ver imagen](#)

El módulo `data_cleaning.py` contiene una función llamada `clean_dataframe` que se centra en mejorar la calidad de los datos para su uso en proyectos, especialmente en contextos donde se requiere un preprocesamiento riguroso, como el entrenamiento de modelos de machine learning. La limpieza de datos es una etapa crucial en cualquier flujo de trabajo de análisis de datos, ya que garantiza que los datos sean precisos, consistentes y estén libres de errores o elementos innecesarios que puedan afectar el rendimiento de los modelos.

La función `clean_dataframe` realiza varias tareas fundamentales para asegurar la calidad de los datos. Primero, permite la eliminación de columnas no deseadas mediante el argumento opcional `columns_to_drop`, que acepta una lista de nombres de columnas a eliminar. Esto es útil para eliminar características irrelevantes o redundantes que no contribuyen al análisis o al entrenamiento del modelo.

Luego, la función maneja los valores faltantes, rellenándolos con la media de las columnas numéricas. Esta estrategia de imputación es simple pero efectiva para evitar problemas que pueden surgir al tener valores nulos, especialmente en algoritmos de aprendizaje automático que no manejan bien los datos incompletos. La media es una elección común ya que minimiza el sesgo que puede introducirse al reemplazar los valores faltantes.

La eliminación de duplicados es otro paso esencial que se realiza, ya que los registros duplicados pueden distorsionar los resultados del análisis y afectar negativamente el rendimiento del modelo al introducir redundancia no deseada. Además, la función también limpia los nombres de las columnas y los datos de caracteres especiales, utilizando expresiones regulares. Esto es especialmente importante en situaciones donde los datos provienen de múltiples fuentes y pueden contener inconsistencias en el formato, lo que puede causar errores durante el procesamiento o el entrenamiento del modelo.

Elección del Algoritmo de Machine Learning

BART (Bidirectional and Auto-Regressive Transformers)

Instanciación del Modelo:

Se ha creado una instancia del modelo utilizando una implementación adecuada proporcionada por la biblioteca correspondiente. Esto asegura una integración eficiente del algoritmo BART.

Entrenamiento:

El modelo se ha entrenado utilizando un conjunto de datos pre entrenamiento mezclado con datos específicos, donde el algoritmo ajusta sus parámetros internos para comprender las preferencias de los usuarios y las características de los productos. Este proceso es esencial para establecer la capacidad generativa del modelo

Descripción del algoritmo

Por medio de un prompt definido por el usuario , en el caso de generar preguntas para la **evaluación técnica**, éste debe especificar para qué cargo o puesto será diseñada la prueba, el nivel de responsabilidad del cargo (senior, middle, junior). Para el caso de **pruebas de habilidades o Soft Skill** es necesario especificar el objetivo de la prueba, perfil del evaluado, nivel de experiencia . Para ambas categorías de evaluación se debe especificar número de preguntas, grado de dificultad, formato de preguntas (opción múltiple, preguntas abiertas, situaciones hipotéticas, escenarios de role-play, etc) , retroalimentación (en caso de ser necesario).

En el caso de la calificación se debe asignar puntajes y ponderaciones, rango y descripción de desempeño por ejemplo, 90-100 = Excelente, 70-89 = Bueno , etc. Se debe establecer metodología de calificación es decir procesamiento de lenguaje natural (NLP) para analizar respuestas abiertas y asignar puntajes. En caso de las habilidades blandas se deben proporcionar normas de evaluación de habilidades específicas, es decir el porcentaje de la habilidad que se está buscando en el aspirante (100% liderazgo, 90% empatía).

Algoritmo programado en Python

```
import torch
from transformers import BartTokenizer, BartForConditionalGeneration, Trainer, TrainingArguments
from datasets import load_dataset

class BARTModel:
    def __init__(self, model_name='facebook/bart-large'):
        self.tokenizer = BartTokenizer.from_pretrained(model_name)
        self.model = BartForConditionalGeneration.from_pretrained(model_name)
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        self.model.to(self.device)

    def generate(self, text, max_length=50):
        inputs = self.tokenizer(text, return_tensors='pt').to(self.device)
        summary_ids = self.model.generate(inputs['input_ids'], max_length=max_length, num_beams=2, early_stopping=True)
        return self.tokenizer.decode(summary_ids[0], skip_special_tokens=True)

    def train(self, train_dataset, val_dataset, output_dir='./results', num_train_epochs=3, per_device_train_batch_size=4):
        training_args = TrainingArguments(
            output_dir=output_dir,
            num_train_epochs=num_train_epochs,
            per_device_train_batch_size=per_device_train_batch_size,
            per_device_eval_batch_size=4,
            evaluation_strategy="epoch",
            save_total_limit=2,
            remove_unused_columns=False,
            load_best_model_at_end=True,
        )
```

[Ver imagen](#)

```
trainer.train()

def preprocess_function(examples, tokenizer, max_input_length=512, max_target_length=128):
    inputs = [doc for doc in examples["article"]]
    model_inputs = tokenizer(inputs, max_length=max_input_length, truncation=True)

    with tokenizer.as_target_tokenizer():
        labels = tokenizer(examples["highlights"], max_length=max_target_length, truncation=True)

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

[Ver imagen](#)

```
def main():
    model_name = 'facebook/bart-large'
    bart_model = BARTModel(model_name=model_name)

    # Cargar el conjunto de datos CNN/DailyMail
    dataset = load_dataset("cnn_dailymail", "3.0.0")
    tokenizer = BartTokenizer.from_pretrained(model_name)

    # Preprocesar el conjunto de datos
    tokenized_datasets = dataset.map(lambda x: preprocess_function(x, tokenizer), batched=True)

    train_dataset = tokenized_datasets["train"].select(range(1000)) # Seleccionamos una pequeña muestra para entrenamiento
    val_dataset = tokenized_datasets["validation"].select(range(500)) # Seleccionamos una pequeña muestra para validación

    # Entrenar el modelo
    bart_model.train(train_dataset, val_dataset)

    # Generar un resumen de ejemplo
    example_text = "El algoritmo BART es muy útil para tareas de procesamiento de lenguaje natural."
    summary = bart_model.generate(example_text)
    print(f"Texto de entrada: {example_text}")
    print(f"Resumen generado: {summary}")

if __name__ == "__main__":
    main()
```

[Ver imagen](#)

Informe de Pruebas Unitarias para el Modelo BART

Objetivo: Evaluar el correcto funcionamiento del modelo BART (Bidirectional and Auto-Regressive Transformers) para tareas de procesamiento de lenguaje natural, asegurando que los resultados sean consistentes y correctos.

Configuración del Entorno de Prueba:

- Se ha utilizado una implementación del modelo BART proporcionada por la biblioteca `transformers` de Hugging Face.
- Se han utilizado datos de ejemplo para probar la carga y el procesamiento de datos, la generación de respuestas y la evaluación del modelo.

Código del Modelo y Pruebas Unitarias

Configuración del Modelo BART

```
1 from transformers import BartTokenizer, BartForConditionalGeneration
2 import torch
3
4 class BARTModel:
5     def __init__(self, model_name='facebook/bart-large'):
6         self.tokenizer = BartTokenizer.from_pretrained(model_name)
7         self.model = BartForConditionalGeneration.from_pretrained(model_name)
8         self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
9         self.model.to(self.device)
10
11     def generate(self, text, max_length=50):
12         inputs = self.tokenizer(text, return_tensors='pt').to(self.device)
13         summary_ids = self.model.generate(inputs['input_ids'], max_length=max_length, num_beams=2, early_stopping=True)
14         return self.tokenizer.decode(summary_ids[0], skip_special_tokens=True)
```

[Ver imagen](#)

Código de Pruebas Unitarias

```
import unittest
import os
from transformers import BartTokenizer, BartForConditionalGeneration
import torch

class TestBARTModel(unittest.TestCase):

    def setUp(self):
        # Crear una instancia del modelo BART
        self.model_name = 'facebook/bart-large'
        self.bart_model = BARTModel(model_name=self.model_name)

    def test_model_initialization(self):
        # Verificar que el modelo y el tokenizer se inicializan correctamente
        self.assertIsInstance(self.bart_model.model, BartForConditionalGeneration, "El modelo no se ha inicializado correctamente")
        self.assertIsInstance(self.bart_model.tokenizer, BartTokenizer, "El tokenizer no se ha inicializado correctamente")

    def test_model_generation(self):
        # Verificar que el modelo puede generar texto correctamente
        input_text = "El algoritmo BART es muy útil para tareas de procesamiento de lenguaje natural."
        output_text = self.bart_model.generate(input_text, max_length=50)
        self.assertIsInstance(output_text, str, "La salida generada no es una cadena de texto.")
        self.assertGreater(len(output_text), 0, "La salida generada está vacía.")

    def test_device_placement(self):
        # Verificar que el modelo se ha colocado en el dispositivo correcto
        self.assertEqual(self.bart_model.model.device, self.bart_model.device, "El modelo no se ha colocado en el dispositivo correcto")

if __name__ == '__main__':
    unittest.main()
```

[Ver imagen](#)

VI. Resultados y discusión

Los resultados obtenidos hasta el momento en las diferentes áreas del proyecto, incluyendo el frontend, el backend y la integración de la inteligencia artificial, muestran el progreso alcanzado en la creación de un sistema que pueda analizar y procesar grandes cantidades de datos de manera efectiva y eficiente.

Front-end:

- La maquetación del frontal ha sido concluida, lo que garantiza la adecuada presentación de la información en el sistema.
- La lógica para conectar con backend las funcionalidades principales ha sido concluida.

IA (Inteligencia Artificial):

- El análisis inicial por parte de los analistas y científicos de datos ha sido realizado.
- Se ha definido la técnica a utilizar para el procesamiento de datos.
- La obtención de datos y su limpieza se han iniciado, lo que permitirá la creación de un conjunto de datos confiables.

Back-end:

- Se ha implementado el CRUD (Create, Read, Update, Delete), lo que permite la gestión eficiente de los datos en la base de datos.
- El proceso de acoplamiento con el front-end y la IA se encuentra en curso.
- Una vez completado el acoplamiento, se comenzará la etapa de pruebas, donde se validará el funcionamiento entre los diferentes componentes del sistema.
- Se han cargado semillas en la base de datos para realizar pruebas y mostrar resultados en el front-end.
- La estructura modular se está implementando, lo que permitirá la reutilización de componentes y módulos.
- Se eliminan las carpetas de models, validators, triggers, schema y se añade la configuración de acl , declarado en el archivo de schema.

En resumen, estos resultados muestran el progreso alcanzado en la creación de un sistema inteligente que combine la potencia de la inteligencia artificial con la flexibilidad y escalabilidad del backend.

VII. Conclusiones y recomendaciones

A pesar de que aún falta el acoplamiento entre cada servicio o microservicio con el backend, los resultados hasta ahora muestran un progreso significativo en la creación del sistema inteligente. El frontend ha sido maquetado y listo para recibir la información procesada por la inteligencia artificial, mientras que el backend está listo para recibir y procesar datos.

Recomendaciones:

- **Continuar con el desarrollo de la IA:** Aprovechar los avances realizados en el análisis inicial y el desarrollo de la técnica de IA para crear un sistema más inteligente y efectivo.
- **Finalizar el acoplamiento entre servicios:** Concentrarse en completar el acoplamiento entre cada servicio o microservicio con el backend, lo que permitirá la creación de un sistema integral y funcional.
- **Realizar pruebas exhaustivas:** Realizar pruebas exhaustivas del sistema para asegurarse de que todos los componentes funcionen correctamente y se integren de manera efectiva.
- **Mejores Prácticas de SEO:** Implementa técnicas de SEO en la aplicación, como el uso de etiquetas meta adecuadas, URLs amigables y optimización del contenido para motores de búsqueda.
- **Implementar Swagger:** Implementar Swagger para documentar los diferentes endpoints del sistema, lo que facilitará la comprensión y uso de los APIs.
- **Monitorear el rendimiento:** Monitorear el rendimiento del sistema y ajustar las configuraciones según sea necesario para asegurarse de que el sistema funcione de manera eficiente y escalable.